

Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральный государственный образовательный университет высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра ТС и ВС

Расчётно-графическая работа

По дисциплине: Базы данных

**Тема: Проектирование и реализация БД
Система мониторинга спутникового интернета — LEO TRACK**

Выполнил: Анисимов Тимур Юрьевич
Группа: ИКС-431

Проверил: Брагин Кирилл Игоревич

1. Описание предметной области

LEO TRACK — реляционная база данных для системы учёта и мониторинга спутникового интернет-сервиса на основе орбитальной группировки низкоорбитальных спутников (LEO — Low Earth Orbit, высота 200–2000 км). Система охватывает полный жизненный цикл абонентского обслуживания: от управления спутниковой инфраструктурой до биллинга и аналитики потребления трафика.

Ключевые сущности и автоматизируемые бизнес-процессы:

Орбитальные плоскости и спутники. Регистрация орбитальных плоскостей (высота, наклонение, проектное количество спутников) и самих спутников с NORAD-идентификатором. Отслеживание статуса каждого аппарата: operational / degraded / standby / decommissioned.

Спутниковые лучи (beams). Каждый спутник формирует несколько узких лучей с заданными координатами центра, полосой пропускания и поляризацией (RHCP / LHCP / LINEAR).

Сервисные планы. Тарифные планы с месячным лимитом трафика, скоростью загрузки/выгрузки, SLA-гарантией аптайма (%) и ценой за сверхлимитный гигабайт.

Абоненты и терминалы. Физические лица и организации, подключённые через наземные спутниковые терминалы. Каждый терминал имеет геолокацию, серийный номер и текущий назначенный луч (beam_coverage).

Сессии передачи данных. Детализация трафика на уровне каждой сессии: байты загрузки и выгрузки, средняя задержка, стоимость сессии.

Телеметрия спутников. Периодические телеметрические пакеты: заряд аккумулятора, мощность солнечных батарей, температура, EIRP нисходящего канала, флаг аномалии.

Биллинг. Ежемесячные счета с базовой суммой по тарифу и надбавкой за превышение квоты трафика. Платежи поддерживают несколько методов: карта, банковский перевод, криптовалюта, ваучер, наличные.

Аудит и архивирование. Автоматическое журналирование всех изменений статуса и баланса абонентов (subscriber_audit), архивирование удалённых счетов (invoice_archive) и лог аномалий телеметрии (satellite_health_log) через триггеры.

2. Скрипт инициализации БД (01_schema.sql)

Единый DDL-файл создаёт 12 основных таблиц. Каждая таблица содержит первичный ключ, внешние ключи с явными правилами CASCADE / RESTRICT / SET NULL, ограничения NOT NULL, UNIQUE и CHECK, реализующие бизнес-правила предметной области. Здесь приведён полный листинг структуры.

01_schema.sql — полный DDL-скрипт структуры базы данных

```
-- ===== ORBITAL PLANES =====
-- Хранит орбитальные плоскости созвездия LEO TRACK.
-- altitude_km: допустимые высоты LEO — 200...2000 км (CHECK).
-- inclination_deg: наклонение 0...180° (CHECK).
CREATE TABLE orbital_planes (
```

```

plane_id SERIAL PRIMARY KEY,
plane_name VARCHAR(50) NOT NULL UNIQUE,
altitude_km INTEGER NOT NULL
CHECK (altitude_km BETWEEN 200 AND 2000),
inclination_deg NUMERIC(6,3) NOT NULL
CHECK (inclination_deg BETWEEN 0 AND 180),
sat_count_design INTEGER NOT NULL CHECK (sat_count_design > 0),
constellation VARCHAR(50) NOT NULL DEFAULT 'LeoTrack-1',
created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- ===== SATELLITES =====
-- Спутники созвездия. norad_id — международный идентификатор (UNIQUE).
-- status: конечный автомат operational/degraded/standby/decommissioned.
-- Уникальность (plane_id, slot_number) — один слот в плоскости.
CREATE TABLE satellites (
sat_id SERIAL PRIMARY KEY,
norad_id VARCHAR(10) NOT NULL UNIQUE,
sat_name VARCHAR(60) NOT NULL,
plane_id INTEGER NOT NULL
REFERENCES orbital_planes(plane_id) ON DELETE RESTRICT,
slot_number INTEGER NOT NULL CHECK (slot_number > 0),
launch_date DATE NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'operational'
CHECK (status IN
('operational','degraded','decommissioned','standby')),
is_active BOOLEAN NOT NULL DEFAULT TRUE,
UNIQUE (plane_id, slot_number)
);

-- ===== SATELLITE BEAMS =====
-- Лучи покрытия каждого спутника.
-- center_lat / center_lon: широта/долгота центра луча с CHECK-диапазонами.
-- polarization: RHCP/LHCP/LINEAR — типы поляризации сигнала.
CREATE TABLE satellite_beams (
beam_id SERIAL PRIMARY KEY,
sat_id INTEGER NOT NULL
REFERENCES satellites(sat_id) ON DELETE CASCADE,
beam_index INTEGER NOT NULL CHECK (beam_index > 0),
center_lat NUMERIC(8,5) NOT NULL
CHECK (center_lat BETWEEN -90 AND 90),
center_lon NUMERIC(8,5) NOT NULL
CHECK (center_lon BETWEEN -180 AND 180),
bandwidth_mhz INTEGER NOT NULL CHECK (bandwidth_mhz > 0),
polarization VARCHAR(4) NOT NULL DEFAULT 'RHCP'
CHECK (polarization IN ('RHCP','LHCP','LINEAR')),
UNIQUE (sat_id, beam_index)
);

```

```

-- ===== SERVICE PLANS =====
-- Тарифные планы. sla_uptime_pct — гарантия доступности (0...100%).
-- price_per_gb — стоимость сверхлимитного трафика.
-- is_active: деактивированные планы защищены триггером от удаления.
CREATE TABLE service_plans (
plan_id SERIAL PRIMARY KEY,
plan_name VARCHAR(80) NOT NULL UNIQUE,
monthly_fee NUMERIC(10,2) NOT NULL CHECK (monthly_fee >= 0),
data_gb_month INTEGER NOT NULL CHECK (data_gb_month > 0),
speed_dl_mbps INTEGER NOT NULL CHECK (speed_dl_mbps > 0),
speed_ul_mbps INTEGER NOT NULL CHECK (speed_ul_mbps > 0),
sla_uptime_pct NUMERIC(5,2) NOT NULL DEFAULT 99.00
CHECK (sla_uptime_pct BETWEEN 0 AND 100),
price_per_gb NUMERIC(8,4) NOT NULL DEFAULT 0.0
CHECK (price_per_gb >= 0),
is_active BOOLEAN NOT NULL DEFAULT TRUE,
created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

-- ===== SUBSCRIBERS =====
-- Абоненты сервиса. email и phone — уникальные идентификаторы клиента.
-- balance: минимум -5000 (CHECK) + защита триггером trg_protect_balance.
-- status: конечный автомат active/blocked/suspended/cancelled.
-- updated_at: автообновление триггером trg_set_updated_at.
CREATE TABLE subscribers (
sub_id SERIAL PRIMARY KEY,
full_name VARCHAR(120) NOT NULL,
email VARCHAR(120) NOT NULL UNIQUE,
phone VARCHAR(20) NOT NULL UNIQUE,
plan_id INTEGER NOT NULL
REFERENCES service_plans(plan_id) ON DELETE RESTRICT,
balance NUMERIC(12,2) NOT NULL DEFAULT 0.00
CHECK (balance >= -5000.00),
status VARCHAR(20) NOT NULL DEFAULT 'active'
CHECK (status IN
('active','blocked','suspended','cancelled')),
registration_date DATE NOT NULL DEFAULT CURRENT_DATE,
passport_number VARCHAR(20),
birth_date DATE,
updated_at TIMESTAMP,
CONSTRAINT chk_email_format CHECK (email LIKE '%@%')
);

-- ===== SUBSCRIBER TERMINALS =====
-- Наземные спутниковые терминалы абонентов.
-- serial_number: заводской номер (UNIQUE).
-- lat/lon: геолокация с валидацией диапазонов CHECK.
CREATE TABLE subscriber_terminals (

```

```

terminal_id SERIAL PRIMARY KEY,
sub_id INTEGER NOT NULL
REFERENCES subscribers(sub_id) ON DELETE CASCADE,
serial_number VARCHAR(30) NOT NULL UNIQUE,
hardware_model VARCHAR(60) NOT NULL,
firmware_ver VARCHAR(20),
lat NUMERIC(8,5) CHECK (lat BETWEEN -90 AND 90),
lon NUMERIC(8,5) CHECK (lon BETWEEN -180 AND 180),
install_date DATE NOT NULL DEFAULT CURRENT_DATE,
status VARCHAR(20) NOT NULL DEFAULT 'online'
C H E C K      ( s t a t u s      I N
('online','offline','faulty','decommissioned'))
);

-- ===== BEAM COVERAGE =====
-- Назначение терминала на спутниковый луч.
-- released_at: момент освобождения; NULL = активное соединение.
-- CHECK гарантирует, что released_at > assigned_at (нет обратного времени).
CREATE TABLE beam_coverage (
coverage_id SERIAL PRIMARY KEY,
terminal_id INTEGER NOT NULL
REFERENCES subscriber_terminals(terminal_id) ON DELETE CASCADE,
beam_id INTEGER NOT NULL
REFERENCES satellite_beams(beam_id) ON DELETE CASCADE,
assigned_at TIMESTAMP NOT NULL DEFAULT NOW(),
released_at TIMESTAMP,
snr_db NUMERIC(6,2),
CONSTRAINT chk_release_after_assign
CHECK (released_at IS NULL OR released_at > assigned_at)
);

-- ===== DATA SESSIONS =====
-- Детализация трафика каждой сессии передачи данных.
-- bytes_down / bytes_up: неотрицательные значения (CHECK).
-- beam_id: может быть NULL (ON DELETE SET NULL), если луч удалён.
CREATE TABLE data_sessions (
session_id BIGSERIAL PRIMARY KEY,
terminal_id INTEGER NOT NULL
REFERENCES subscriber_terminals(terminal_id) ON DELETE CASCADE,
beam_id INTEGER
REFERENCES satellite_beams(beam_id) ON DELETE SET NULL,
start_time TIMESTAMP NOT NULL,
end_time TIMESTAMP,
bytes_down BIGINT NOT NULL DEFAULT 0 CHECK (bytes_down >= 0),
bytes_up BIGINT NOT NULL DEFAULT 0 CHECK (bytes_up >= 0),
avg_latency_ms INTEGER CHECK (avg_latency_ms >= 0),
session_cost NUMERIC(10,4) NOT NULL DEFAULT 0.0000
CHECK (session_cost >= 0),

```

```

CONSTRAINT chk_end_after_start
CHECK (end_time IS NULL OR end_time > start_time)
);

-- ===== TELEMETRY =====
-- Телеметрические пакеты со спутника.
-- anomaly_flag: при TRUE триггер записывает в satellite_health_log.
CREATE TABLE telemetry (
  tlm_id BIGSERIAL PRIMARY KEY,
  sat_id INTEGER NOT NULL
  REFERENCES satellites(sat_id) ON DELETE CASCADE,
  recorded_at TIMESTAMP NOT NULL DEFAULT NOW(),
  battery_pct NUMERIC(5,2) CHECK (battery_pct BETWEEN 0 AND 100),
  solar_watts NUMERIC(8,2) CHECK (solar_watts >= 0),
  temp_c NUMERIC(6,2),
  downlink_eirp NUMERIC(7,3),
  anomaly_flag BOOLEAN NOT NULL DEFAULT FALSE
);

-- ===== INVOICES =====
-- Ежемесячные счета абонентам.
-- UNIQUE (sub_id, billing_period): один счёт на абонента в месяц.
-- amount_overage: сверхлимитная надбавка, пересчитывается функцией.
-- Триггер trg_invoice_status_guard запрещает откат статуса paid→pending.
CREATE TABLE invoices (
  invoice_id SERIAL PRIMARY KEY,
  sub_id INTEGER NOT NULL
  REFERENCES subscribers(sub_id) ON DELETE CASCADE,
  billing_period VARCHAR(7) NOT NULL, -- формат 'YYYY-MM'
  issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
  due_date DATE NOT NULL,
  amount_base NUMERIC(10,2) NOT NULL CHECK (amount_base >= 0),
  amount_overage NUMERIC(10,2) NOT NULL DEFAULT 0.00
  CHECK (amount_overage >= 0),
  total_amount NUMERIC(10,2) NOT NULL CHECK (total_amount >= 0),
  status VARCHAR(20) NOT NULL DEFAULT 'pending'
  CHECK (status IN
('pending','paid','overdue','cancelled')),
  UNIQUE (sub_id, billing_period),
  CONSTRAINT chk_due_after_issue CHECK (due_date >= issue_date)
);

-- ===== PAYMENTS =====
-- Платежи по счетам. transaction_ref: уникальная ссылка транзакции.
-- method: допустимые методы оплаты (CHECK enum).
-- invoice_id: ON DELETE SET NULL — платёж сохраняется при удалении счёта.
CREATE TABLE payments (
  payment_id SERIAL PRIMARY KEY,
  sub_id INTEGER NOT NULL

```

```

REFERENCES subscribers(sub_id) ON DELETE RESTRICT,
invoice_id INTEGER
REFERENCES invoices(invoice_id) ON DELETE SET NULL,
amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
payment_date TIMESTAMP NOT NULL DEFAULT NOW(),
method VARCHAR(30) NOT NULL DEFAULT 'card'
CHECK (method IN
('card','bank_transfer','crypto','voucher','cash')),
transaction_ref VARCHAR(60) NOT NULL UNIQUE
);

-- ===== PLAN CHANGES =====
-- История смены тарифных планов абонентом.
-- old_plan_id: ON DELETE SET NULL — история не теряется при удалении плана.
CREATE TABLE plan_changes (
change_id SERIAL PRIMARY KEY,
sub_id INTEGER NOT NULL
REFERENCES subscribers(sub_id) ON DELETE CASCADE,
old_plan_id INTEGER REFERENCES service_plans(plan_id) ON DELETE SET NULL,
new_plan_id INTEGER NOT NULL
REFERENCES service_plans(plan_id) ON DELETE RESTRICT,
changed_at TIMESTAMP NOT NULL DEFAULT NOW(),
reason TEXT
);

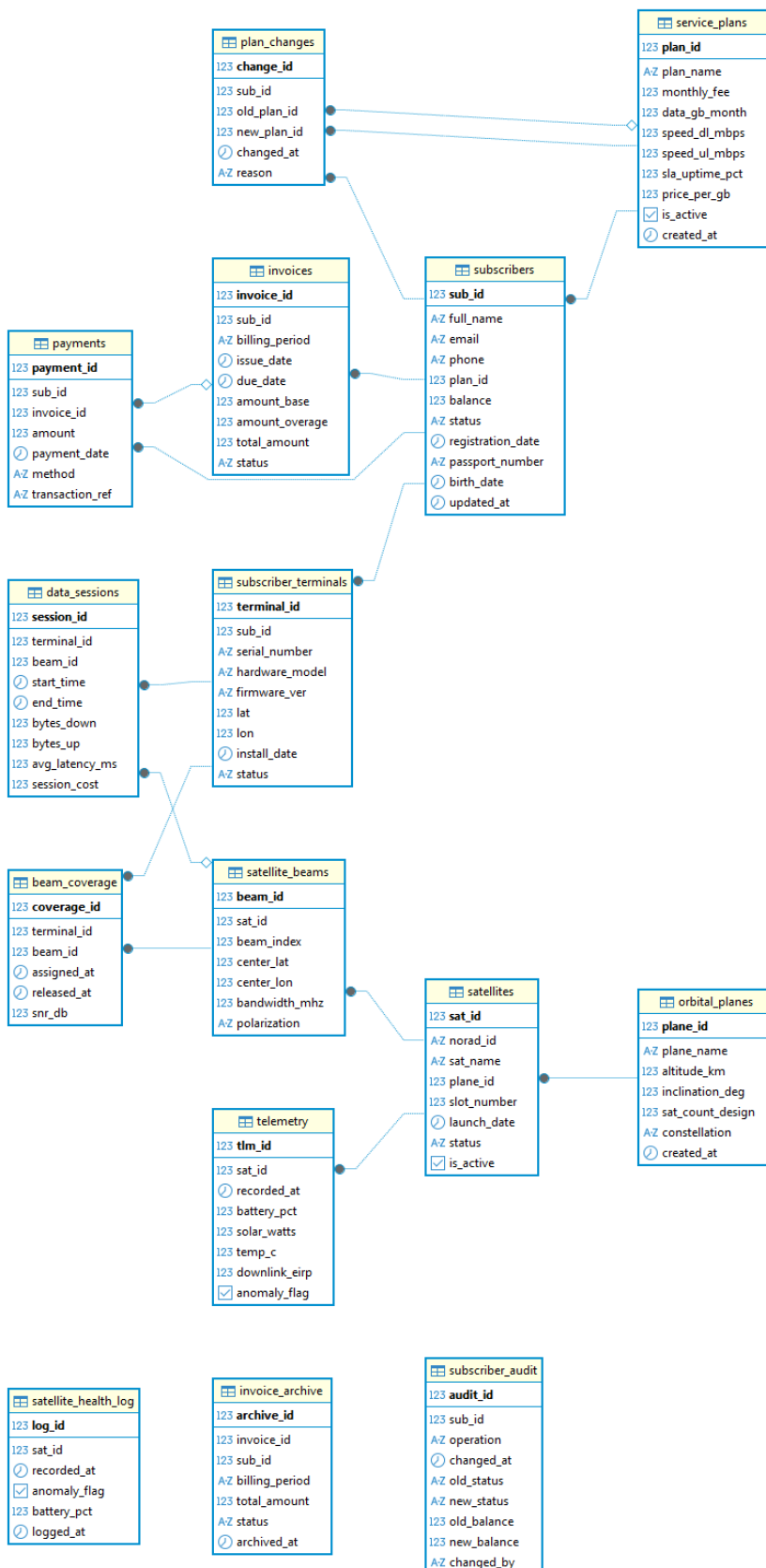
-- ===== ИНДЕКСЫ =====
-- Индексы на поля с высокой селективностью и частым использованием
-- в условиях WHERE, JOIN и ORDER BY.
CREATE INDEX idx_satellites_plane ON satellites (plane_id);
CREATE INDEX idx_satellites_status ON satellites (status);
CREATE INDEX idx_beams_sat ON satellite_beams (sat_id);
CREATE INDEX idx_subs_plan ON subscribers (plan_id);
CREATE INDEX idx_subs_status ON subscribers (status);
CREATE INDEX idx_subs_balance ON subscribers (balance);
CREATE INDEX idx_terminals_sub ON subscriber_terminals(sub_id);
CREATE INDEX idx_sessions_terminal ON data_sessions (terminal_id);
CREATE INDEX idx_sessions_start ON data_sessions (start_time DESC);
-- Составной индекс: быстрая выборка последних сессий по терминалу
CREATE INDEX idx_sessions_terminal_start ON data_sessions
(terminal_id, start_time DESC);
-- Составной: телеметрия конкретного спутника за период
CREATE INDEX idx_tlm_sat_time ON telemetry
(sat_id, recorded_at DESC);
-- Частичный (partial) индекс: только аномальные записи → экономия места
CREATE INDEX idx_tlm_anomaly ON telemetry (anomaly_flag)
WHERE anomaly_flag = TRUE;
CREATE INDEX idx_inv_sub ON invoices (sub_id);
CREATE INDEX idx_inv_status ON invoices (status);

```

```
CREATE INDEX idx_inv_period ON invoices (billing_period);
CREATE INDEX idx_pay_sub ON payments (sub_id);
CREATE INDEX idx_pay_date ON payments (payment_date DESC);
```

3. Графическая модель данных (ER-диаграмма)

ER-диаграмма отражает логическую структуру базы данных LEO TRACK: 12 основных таблиц, связанных внешними ключами. Диаграмма сгенерирована в DBeaver



Основные связи между таблицами:

Таблица-родитель → Дочерняя	Тип	Описание
orbital_planes → satellites	1:N	Один орбитальный план содержит несколько спутников. ON DELETE RESTRICT — нельзя удалить план с активными спутниками
satellites → satellite_beams	1:N	Каждый спутник формирует несколько лучей. ON DELETE CASCADE — при удалении спутника удаляются все его лучи
satellite_beams → beam_coverage	1:N	Луч может обслуживать несколько терминалов. released_at IS NULL = активное назначение
satellite_beams → data_sessions	1:N	Сессия работает через конкретный луч. beam_id nullable (ON DELETE SET NULL) — сессия сохраняется при удалении луча
satellites → telemetry	1:N	Телеметрические данные привязаны к спутнику. ON DELETE CASCADE
service_plans → subscribers	1:N	Абонент подключён к одному тарифному плану. ON DELETE RESTRICT — план нельзя удалить при живых абонентах
subscribers → subscriber_terminals	1:N	У абонента может быть несколько терминалов. ON DELETE CASCADE
subscriber_terminals → data_sessions	1:N	Сессии регистрируются по конкретному терминалу
subscribers → invoices	1:N	Счета выставляются абоненту ежемесячно. UNIQUE(sub_id, billing_period) — один счёт в месяц
invoices → payments	1:N	К счёту может быть несколько платежей. invoice_id ON DELETE SET NULL
subscribers → plan_changes	1: N	Полная история смены тарифных планов с датой и причиной

4. Наполнение базы данных (02_data.sql)

Скрипт вставляет минимальный репрезентативный набор данных для проверки работоспособности всех запросов. Суммарно более 100 записей:

Таблица	Записей	Описание
orbital_planes	5	Орбитальные плоскости (Alpha-550 ... Epsilon-590)
satellites	10	Спутники созвездия с NORAD ID
satellite_beams	10	Лучи покрытия (луч на крупные города мира)
service_plans	7	Тарифы от Explorer (29.99\$/мес) до Government (799.99\$)
subscribers	12	Абоненты: активные, заблокированные, с задолженностью
subscriber_terminals	12	Терминалы с геолокацией
beam_coverage	10	Активные назначения терминалов на лучи
data_sessions	17	Сессии за март–май 2026 с реальными объемами байт
telemetry	10+	Телеметрия с аномалиями (sat_id 3 и 6)
invoices	24	Счета за 3 периода: оплаченные, pending, overdue
payments	16	Платежи разными методами (card/bank/crypto/voucher)
plan_changes	4	История смены тарифов с причинами

02_data.sql — фрагмент вставки тестовых данных

```
-- Тарифные планы — от базового до государственного
INSERT INTO service_plans
(plan_name, monthly_fee, data_gb_month, speed_dl_mbps,
speed_ul_mbps, sla_uptime_pct, price_per_gb)
VALUES
('Explorer', 29.99, 50, 50, 10, 99.00, 0.50),
('Nomad', 59.99, 150, 100, 20, 99.50, 0.40),
('Professional', 149.99, 500, 200, 50, 99.90, 0.30),
('Enterprise', 399.99, 2000, 500, 100, 99.99, 0.20),
('Maritime', 299.99, 1000, 300, 75, 99.80, 0.25),
('Aviation', 499.99, 3000, 600, 120, 99.99, 0.15),
('Government', 799.99, 9999, 800, 200, 99.99, 0.10);

-- Абоненты: активные, заблокированные, с отрицательным балансом
INSERT INTO subscribers
(full_name, email, phone, plan_id, balance, status, registration_date)
VALUES
('Anisimov Timur Yurievich', 'tanisimov2006@gmail.com',
'+79991234560', 7, 9999.00, 'active', '2023-01-01'),
```

```
('Elena Volkova', 'e.volkova@company.ru',
'+79161001001', 3, 450.00, 'active', '2023-03-15'),
('Ivan Sidorov', 'isidorov@fishery.ru',
'+79164004004', 5, -50.00, 'blocked', '2023-06-20'),
('Mikhail Zaitsev', 'mzaitsev@mine.corp',
'+79160020020', 5, -380.00, 'blocked', '2024-03-19');
-- ... всего 12 записей
```

5. Ограничения целостности и стратегия индексации

Обоснование ограничений CHECK и UNIQUE

Каждое ограничение отражает реальное бизнес-правило предметной области и предотвращает некорректные данные на уровне СУБД:

Ограничение	Обоснование
CHECK (altitude km BETWEEN 200 AND 2000)	Физически допустимый диапазон высот LEO-орбит в километрах
CHECK (balance >= -5000.00)	Максимально допустимый долг абонента; дополнительно защищён триггером
CHECK (status IN (...))	Конечный автомат статусов — только предопределённые значения
CHECK (sla_uptime_pct BETWEEN 0 AND 100)	Гарантия доступности не может быть вне диапазона 0–100%
CHECK (due_date >= issue_date)	Срок оплаты счёта не может быть раньше даты выставления
CHECK (end_time > start_time)	Конец сессии передачи данных строго после её начала
UNIQUE (email), UNIQUE (phone)	Абонент идентифицируется по уникальным контактным данным
UNIQUE (serial_number)	Заводской серийный номер оборудования уникален глобально
UNIQUE (sub_id, billing_period)	Один счёт на абонента в месяц — предотвращает двойное выставление
UNIQUE (norad_id)	Международный идентификатор спутника уникален в мировом реестре

Обоснование индексов

Индексы создаются исключительно на полях с высокой селективностью и доказанным частым использованием в WHERE, JOIN и ORDER BY. Избыточные индексы замедляют INSERT/UPDATE/DELETE и тратят дисковое пространство.

Индекс(ы)	Таблица / поле	Обоснование
idx_subs_plan idx_subs_status idx_subs_balance	subscribers (plan_id / status / balance)	Три самых частых фильтра: JOIN по плану, выборка активных/заблокированных, поиск должников (balance < 0)
idx_sessions_terminal_start	data_sessions (terminal_id, start_time DES)	Составной индекс под запрос «последние сессии терминала» — устраняет дополнительную сортировку (Sort-узел исчезает из плана EXPLAIN)
idx_tlm_sat_time	telemetry (sat_id, recorded_at DESC)	Составной индекс для аналитики телеметрии конкретного спутника за временной период

Индекс(ы)	Таблица / поле	Обоснование
idx_tlm_anomaly	telemetry(anomaly_flag) WHERE anomaly_flag = TRU	Частичный (partial) индекс — хранит только аномальные записи (≈3% строк), минимальный overhead при INSERT
idx_inv_status idx_inv_period	invoices (status / billing_period)	Биллинговые выборки: фильтрация неоплаченных счетов и группировка по расчётным периодам
idx_pay_date	payments (payment_date DESC)	Сортировка «последние платежи» без полного сканирования таблицы

6. Аналитические SQL-запросы (05_queries.sql)

Набор из 10 запросов покрывает все обязательные конструкции: простые SELECT, WHERE, JOIN, GROUP BY, HAVING, ORDER BY/LIMIT, CTE (WITH), подзапросы и аналитику по временным периодам. Каждый запрос решает реальную задачу из предметной области системы LEO TRACK.

6.1. Реестр абонентов с тарифным планом

Тип запроса: `SELECT + JOIN`

Что делает этот запрос:

Выводит полный список абонентов: имя, контакты, название тарифного плана, ежемесячная стоимость, баланс, статус и дата регистрации. Используется оператором как основная рабочая таблица клиентов.

Зачем выбраны именно эти конструкции:

JOIN с таблицей service_plans необходим, чтобы вместо числового plan_id показывать читаемое название тарифа. Без JOIN пришлось бы делать отдельный запрос для каждого абонента (N+1 проблема).

SQL — запрос 6.1

```
SELECT
  s.sub_id,
  s.full_name,
  s.email,
  s.phone,
  sp.plan_name, -- название тарифа вместо числового plan_id
  sp.monthly_fee,
  s.balance,
  s.status,
  s.registration_date
FROM subscribers s
JOIN service_plans sp ON sp.plan_id = s.plan_id
ORDER BY s.sub_id;
```

```

leotrack=> SELECT
leotrack->   s.sub_id,
leotrack->   s.full_name,
leotrack->   s.email,
leotrack->   s.phone,
leotrack->   sp.plan_name, -- название тарифа вместо числового plan_id
leotrack->   sp.monthly_fee,
leotrack->   s.balance,
leotrack->   s.status,
leotrack->   s.registration_date
leotrack-> FROM subscribers s
leotrack-> JOIN service_plans sp ON sp.plan_id = s.plan_id
leotrack-> ORDER BY s.sub_id;

```

sub_id	full_name	email	phone	plan_name	monthly_fee	balance	status	registration_date
1	Anisimov Timur Yurievich	tanisimov2006@gmail.com	+79991234560	Government	799.99	9999.00	active	2023-01-01
2	Elena Volkova	e.volkova@company.ru	+79161001001	Professional	149.99	450.00	active	2023-03-15
3	Sergey Petrov	spetrov@logistic.com	+79162002002	Nomad	59.99	720.50	active	2023-04-10
4	Maria Ivanova	maria.i@enterprise.ru	+79163003003	Enterprise	399.99	2100.00	active	2023-05-01
5	Ivan Sidorov	isidorov@fishery.ru	+79164004004	Maritime	299.99	-50.00	blocked	2023-06-20
6	Nikolai Frolov	nfrolov@aviation.aero	+79165005005	Aviation	499.99	1500.00	active	2023-07-08
7	Anna Morozova	anna.m@startup.io	+79166006006	Explorer	29.99	15.00	active	2023-08-14
8	Dmitry Kuznetsov	dkuznetsov@geo.survey	+79167007007	Professional	149.99	800.00	active	2023-09-02
9	Olga Smirnova	osmirnova@ngo.org	+79168008008	Nomad	59.99	0.00	active	2023-10-17
10	Alexei Novikov	anovikov@research.edu	+79169009009	Professional	149.99	320.00	active	2024-01-05
11	Yuliya Belova	y.belova@media.tv	+79160010010	Enterprise	399.99	1200.00	active	2024-02-11
12	Mikhail Zaitsev	mzaitsev@mine.corp	+79160020020	Maritime	299.99	-380.00	blocked	2024-03-19
13	Alexander Vinogradov	user1@leotrack.io	+79001000001	Explorer	29.99	173.17	act— More —	

Рис. Результат выполнения запроса 6.1

6.2. Должники — абоненты с отрицательным балансом

Тип запроса: `WHERE + JOIN + ORDER BY`

Что делает этот запрос:

Выбирает абонентов, у которых баланс ушёл в минус, с указанием текущего тарифа. Используется отделом по работе с задолженностями для формирования списка звонков и блокировок.

Зачем выбраны именно эти конструкции:

`WHERE s.balance < 0` фильтрует только должников. `ORDER BY s.balance ASC` ставит наиболее критичные случаи (самый большой долг) в начало списка — без сортировки оператор тратил бы время на ручной поиск приоритетов.

SQL — запрос 6.2

```

SELECT
  s.sub_id,
  s.full_name,
  s.phone,
  s.balance, -- отрицательные значения: долг абонента
  s.status,
  sp.plan_name
FROM subscribers s
JOIN service_plans sp ON sp.plan_id = s.plan_id
WHERE s.balance < 0 -- только должники
ORDER BY s.balance ASC; -- сначала наибольший долг

```

```

leotrack=> SELECT
leotrack->     s.sub_id,
leotrack->     s.full_name,
leotrack->     s.phone,
leotrack->     s.balance, -- отрицательные значения: долг абонента
leotrack->     s.status,
leotrack->     sp.plan_name
leotrack-> FROM subscribers s
leotrack-> JOIN service_plans sp ON sp.plan_id = s.plan_id
leotrack-> WHERE s.balance < 0 -- только должники
leotrack-> ORDER BY s.balance ASC; -- сначала наибольший долг

```

sub_id	full_name	phone	balance	status	plan_name
12	Mikhail Zaitsev	+79160020020	-380.00	blocked	Maritime
5	Ivan Sidorov	+79164004004	-50.00	blocked	Maritime

(2 rows)

Рис. Результат выполнения запроса 6.2

6.3. Финансовый отчёт по расчётным периодам

Тип запроса: `GROUP BY + SUM + CASE WHEN`

Что делает этот запрос:

Агрегирует выставленные счета по месяцам: суммарный биллинг, собранная выручка и непогашенная задолженность. Формирует ежемесячный финансовый отчёт для руководства оператора.

Зачем выбраны именно эти конструкции:

`GROUP BY billing_period` группирует строки по месяцу. `SUM + CASE WHEN` — условная агрегация: разбивает общую сумму на оплаченную и неоплаченную части без дополнительных подзапросов, что эффективнее двух отдельных `SELECT`.

SQL – запрос 6.3

```

SELECT
i.billing_period,
COUNT(*) AS invoices_total,
SUM(i.total_amount) AS billed_total,
-- условная сумма: только оплаченные счета
SUM(CASE WHEN i.status = 'paid'
THEN i.total_amount ELSE 0 END) AS collected,
-- условная сумма: неоплаченные и просроченные
SUM(CASE WHEN i.status IN ('pending','overdue')
THEN i.total_amount ELSE 0 END) AS outstanding
FROM invoices i
GROUP BY i.billing_period
ORDER BY i.billing_period;

```

```

leotrack=> SELECT
leotrack->     i.billing_period,
leotrack->     COUNT(*) AS invoices_total,
leotrack->     SUM(i.total_amount) AS billed_total,
leotrack->     -- условная сумма: только оплаченные счета
leotrack->     SUM(CASE WHEN i.status = 'paid'
leotrack(>         THEN i.total_amount ELSE 0 END) AS collected,
leotrack->     -- условная сумма: неоплаченные и просроченные
leotrack->     SUM(CASE WHEN i.status IN ('pending','overdue')
leotrack(>         THEN i.total_amount ELSE 0 END) AS outstanding
leotrack-> FROM invoices i
leotrack-> GROUP BY i.billing_period
leotrack-> ORDER BY i.billing_period;
  billing_period | invoices_total | billed_total | collected | outstanding
-----+-----+-----+-----+-----
  2026-03        |          60    |  26160.31   |  15553.99 |  10606.32
  2026-04        |          61    |  26461.84   |  15390.02 |  11071.82
  2026-05        |          59    |  18529.41   |  11379.66 |   7149.75
(3 rows)

```

Рис. Результат выполнения запроса 6.3

6.4. Сводная карточка абонента

Тип запроса: `LEFT JOIN + COUNT(DISTINCT) + COALESCE`

Что делает этот запрос:

Формирует сводный отчёт по каждому абоненту: тарифный план, текущий баланс, количество терминалов и суммарные платежи за всё время обслуживания.

Зачем выбраны именно эти конструкции:

LEFT JOIN гарантирует включение абонентов без платежей и без терминалов — с INNER JOIN такие строки потерялись бы. COUNT(DISTINCT) исключает дублирование из-за декартового произведения при нескольких JOIN. COALESCE(..., 0) заменяет NULL на 0 для абонентов без платежей.

SQL — запрос 6.4

```
SELECT
  s.sub_id,
  s.full_name,
  sp.plan_name,
  sp.monthly_fee,
  s.balance,
  s.status,
  COUNT(DISTINCT t.terminal_id) AS terminals,
  COALESCE(SUM(p.amount), 0) AS total_paid_all_time
FROM subscribers s
JOIN service_plans sp ON sp.plan_id = s.plan_id
LEFT JOIN subscriber_terminals t ON t.sub_id = s.sub_id
LEFT JOIN payments p ON p.sub_id = s.sub_id
GROUP BY
  s.sub_id, s.full_name, sp.plan_name,
  sp.monthly_fee, s.balance, s.status
ORDER BY total_paid_all_time DESC;
```



```

leotrack=> SELECT
leotrack->     s.sub_id,
leotrack->     s.full_name,
leotrack->     sp.plan_name,
leotrack->     sp.monthly_fee,
leotrack->     s.balance,
leotrack->     s.status,
leotrack->     COUNT(DISTINCT t.terminal_id) AS terminals,
leotrack->     COALESCE(SUM(p.amount), 0) AS total_paid_all_time
leotrack-> FROM subscribers s
leotrack-> JOIN service_plans sp ON sp.plan_id = s.plan_id
leotrack-> LEFT JOIN subscriber_terminals t ON t.sub_id = s.sub_id
leotrack-> LEFT JOIN payments p ON p.sub_id = s.sub_id
leotrack-> GROUP BY
leotrack->     s.sub_id, s.full_name, sp.plan_name,
leotrack->     sp.monthly_fee, s.balance, s.status
leotrack-> ORDER BY total_paid_all_time DESC;

```

sub_id	full_name	plan_name	monthly_fee	balance	status	terminals	total_paid_all_time
1	Anisimov Timur Yurievich	Government	799.99	9999.00	active	1	4088.75
4	Maria Ivanova	Enterprise	399.99	2100.00	active	1	3276.48
6	Nikolai Frolov	Aviation	499.99	1500.00	active	1	3117.29
33	James Morozov	Government	799.99	1636.57	cancelled	1	2490.87
11	Yuliya Belova	Enterprise	399.99	1200.00	active	1	2472.49
47	Ava Clark	Government	799.99	2660.95	cancelled	1	2198.07
26	Olga Huang	Government	799.99	1124.38	cancelled	1	2073.72
61	Luca Johnson	Government	799.99	3685.33	cancelled	1	1912.47
12	Mikhail Zaitsev	Maritime	299.99	-380.00	blocked	1	1833.75
40	Joseph Sidorov	Government	799.99	2148.76	cancelled	1	1773.82
53	Wei Harris	Aviation	499.99	3099.97	suspended	1	1654.87
18	Mikhail Zaitsev	Aviation	499.99	539.02	suspended	1	1621.62
58	Fatima Taylor	Enterprise	399.99	3465.82	active	1	1612.12
54	Li White	Government	799.99	3173.14	cancelled	1	1481.12
19	Ivan Smirnov	Government	799.99	612.19	cancelled	1	1461.87
32	Yulia Chen	Aviation	499.99	1563.40	suspended	1	1420.32
23	Maria Zhou	Enterprise	399.99	904.87	active	1	1341.37
46	Isabella Rodriguez	Aviation	499.99	2587.78	suspended	1	1226.22
51	Harper Thompson	Enterprise	399.99	2953.63	active	1	1192.47
25	Anna Zhao	Aviation	499.99	1051.21	suspended	1	1179.07
37	William Sokolov	Enterprise	399.99	1929.25	active	1	1169.57
38	David Popov	Maritime	299.99	2002.42	blocked	1	1070.62
60	Sofia Brown	Aviation	499.99	3612.16	suspended	1	1039.32
16	Andrey Abramov	Enterprise	399.99	392.68	active	1	978.72
5	Ivan Sidorov	Maritime	299.99	-50.00	blocked	1	978.48
39	Richard Kuznetsov	Aviation	499.99	2075.59	suspended	1	977.87
8	Dmitry Kuznetsov	Professional	149.99	800.00	active	1	973.98
44	Olivia Lee	Enterprise	399.99	2441.44	active	1	953.12

Рис. Результат выполнения запроса 6.4

6.5. Прибыльность тарифных планов — только высокий средний чек

Тип запроса: `GROUP BY + HAVING + AVG`

Что делает этот запрос:

Анализирует прибыльность каждого тарифного плана по оплаченным счетам. Оставляет только планы с высоким средним чеком (более 200\$). Используется маркетинговым отделом.

Зачем выбраны именно эти конструкции:

`HAVING AVG(i.total_amount) > 200` — это фильтр по агрегированному значению. В отличие от `WHERE`, который фильтрует строки ДО группировки, `HAVING` работает ПОСЛЕ: сначала считается средний чек по каждому плану, и только потом отбрасываются планы с низким значением.

SQL — запрос 6.5

```
SELECT
  sp.plan_name,
  COUNT(DISTINCT s.sub_id) AS subscriber_count,
  ROUND(AVG(i.total_amount), 2) AS avg_invoice_amount,
  SUM(i.total_amount) AS total_revenue
FROM service_plans sp
JOIN subscribers s ON s.plan_id = sp.plan_id
JOIN invoices i ON i.sub_id = s.sub_id
WHERE i.status = 'paid' -- только оплаченные счета
GROUP BY sp.plan_id, sp.plan_name
HAVING AVG(i.total_amount) > 200 -- фильтр по агрегату — только ПОСЛЕ GROUP BY
ORDER BY avg_invoice_amount DESC;
```

```
leotrack=> SELECT
leotrack->   sp.plan_name,
leotrack->   COUNT(DISTINCT s.sub_id) AS subscriber_count,
leotrack->   ROUND(AVG(i.total_amount), 2) AS avg_invoice_amount,
leotrack->   SUM(i.total_amount) AS total_revenue
leotrack-> FROM service_plans sp
leotrack-> JOIN subscribers s ON s.plan_id = sp.plan_id
leotrack-> JOIN invoices i ON i.sub_id = s.sub_id
leotrack-> WHERE i.status = 'paid' -- только оплаченные счета
leotrack-> GROUP BY sp.plan_id, sp.plan_name
leotrack-> HAVING AVG(i.total_amount) > 200 -- фильтр по агрегату — только ПОСЛЕ GROUP BY
leotrack-> ORDER BY avg_invoice_amount DESC;
```

plan_name	subscriber_count	avg_invoice_amount	total_revenue
Government	8	877.85	12289.86
Aviation	8	582.28	8151.94
Enterprise	9	494.55	7912.73
Maritime	8	354.85	5322.76
Professional	10	266.21	3993.08

(5 rows)

Рис. Результат выполнения запроса 6.5

6.6. Превышение квоты трафика за прошлый месяц

Тип запроса: CTE (WITH) + DATE_TRUNC

Что делает этот запрос:

Находит абонентов, потребивших более 80% своей месячной квоты в предыдущем календарном месяце. Список используется для проактивного предложения апгрейда тарифа или принудительного ограничения скорости.

Зачем выбраны именно эти конструкции:

WITH session_totals AS (...) — обобщённое табличное выражение (CTE). Выносит сложный подсчёт трафика в отдельный именованный блок, делая основной запрос читаемым. DATE_TRUNC("month", ...) обрезает метку времени до начала месяца — стандартный способ выборки по календарным периодам в PostgreSQL.

SQL — запрос 6.6

```
-- CTE: суммарный трафик каждого абонента за прошлый месяц
WITH session_totals AS (
  SELECT
    t.sub_id,
    -- перевод байт в гигабайты
    SUM(ds.bytes_down + ds.bytes_up) / 1073741824.0 AS used_gb
  FROM data_sessions ds
  JOIN subscriber_terminals t ON t.terminal_id = ds.terminal_id
  -- DATE_TRUNC('month', ...) = начало предыдущего месяца
  WHERE ds.start_time >= DATE_TRUNC('month',
    CURRENT_DATE - INTERVAL '1 month')
  AND ds.start_time < DATE_TRUNC('month', CURRENT_DATE)
  GROUP BY t.sub_id
)
-- Основной запрос: фильтруем тех, кто превысил 80% квоты
SELECT
  s.sub_id,
  s.full_name,
  sp.plan_name,
  sp.data_gb_month AS plan_quota_gb,
  ROUND(st.used_gb::NUMERIC, 2) AS used_gb,
  ROUND((st.used_gb / sp.data_gb_month
    * 100)::NUMERIC, 1) AS usage_pct
FROM session_totals st
JOIN subscribers s ON s.sub_id = st.sub_id
JOIN service_plans sp ON sp.plan_id = s.plan_id
WHERE st.used_gb > sp.data_gb_month * 0.80 -- >80% квоты
ORDER BY usage_pct DESC;
```

```

leotrack=> -- CTE: суммарный трафик каждого абонента за прошлый месяц
leotrack=> WITH session_totals AS (
leotrack(>     SELECT
leotrack(>         t.sub_id,
leotrack(>         -- перевод байт в гигабайты
leotrack(>         SUM(ds.bytes_down + ds.bytes_up) / 1073741824.0 AS used_gb
leotrack(>     FROM data_sessions ds
leotrack(>     JOIN subscriber_terminals t ON t.terminal_id = ds.terminal_id
leotrack(>     -- DATE_TRUNC('month', ...) = начало предыдущего месяца
leotrack(>     WHERE ds.start_time >= DATE_TRUNC('month',
leotrack(>         CURRENT_DATE - INTERVAL '1 month')
leotrack(>     AND ds.start_time < DATE_TRUNC('month', CURRENT_DATE)
leotrack(>     GROUP BY t.sub_id
leotrack(> )
leotrack-> -- Основной запрос: фильтруем тех, кто превысил 80% квоты
leotrack-> SELECT
leotrack->     s.sub_id,
leotrack->     s.full_name,
leotrack->     sp.plan_name,
leotrack->     sp.data_gb_month AS plan_quota_gb,
leotrack->     ROUND(st.used_gb::NUMERIC, 2) AS used_gb,
leotrack->     ROUND((st.used_gb / sp.data_gb_month
leotrack->         * 100)::NUMERIC, 1) AS usage_pct
leotrack-> FROM session_totals st
leotrack-> JOIN subscribers s ON s.sub_id = st.sub_id
leotrack-> JOIN service_plans sp ON sp.plan_id = s.plan_id
leotrack-> WHERE st.used_gb > sp.data_gb_month * 0.80 -->80% квоты
leotrack-> ORDER BY usage_pct DESC;
  sub_id | full_name | plan_name | plan_quota_gb | used_gb | usage_pct
-----+-----+-----+-----+-----+-----
(0 rows)

```

Рис. Результат выполнения запроса 6.6

6.7. Топ-5 спутников по аномалиям за последние 30 дней

Тип запроса: `COUNT FILTER + AVG + LIMIT`

Что делает этот запрос:

Показывает топ-5 спутников с наибольшим числом телеметрических аномалий за последние 30 дней. Используется дежурным инженером для мониторинга здоровья созвездия.

Зачем выбраны именно эти конструкции:

`COUNT(*) FILTER (WHERE anomaly_flag = TRUE)` — встроенный синтаксис PostgreSQL для условного подсчёта (аналог `SUM(CASE WHEN)`), но компактнее). `LIMIT 5` ограничивает выдачу топом без дополнительных подзапросов. Сортировка сначала по аномалиям, затем по заряду батареи — составной приоритет критичности.

SQL — запрос 6.7

```
SELECT
  sat.sat_id,
  sat.norad_id,
  sat.sat_name,
  sat.status,
  op.plane_name,
  -- условный подсчёт аномалий без дополнительного подзапроса
  COUNT(*) FILTER (WHERE tlm.anomaly_flag = TRUE) AS anomaly_count,
  ROUND(AVG(tlm.battery_pct), 1) AS avg_battery_pct
FROM satellites sat
JOIN orbital_planes op ON op.plane_id = sat.plane_id
JOIN telemetry tlm ON tlm.sat_id = sat.sat_id
WHERE tlm.recorded_at >= NOW() - INTERVAL '30 days'
GROUP BY sat.sat_id, sat.norad_id,
  sat.sat_name, sat.status, op.plane_name
ORDER BY anomaly_count DESC,
  avg_battery_pct ASC -- при равных аномалиях — сначала худший заряд
LIMIT 5;
```

```

leotrack=> SELECT
leotrack->     sat.sat_id,
leotrack->     sat.norad_id,
leotrack->     sat.sat_name,
leotrack->     sat.status,
leotrack->     op.plane_name,
leotrack->     -- условный подсчёт аномалий без дополнительного подзапроса
leotrack->     COUNT(*) FILTER (WHERE tlm.anomaly_flag = TRUE) AS anomaly_count,
leotrack->     ROUND(AVG(tlm.battery_pct), 1) AS avg_battery_pct
leotrack-> FROM satellites sat
leotrack-> JOIN orbital_planes op ON op.plane_id = sat.plane_id
leotrack-> JOIN telemetry tlm ON tlm.sat_id = sat.sat_id
leotrack-> WHERE tlm.recorded_at >= NOW() - INTERVAL '30 days'
leotrack-> GROUP BY sat.sat_id, sat.norad_id,
leotrack->     sat.sat_name, sat.status, op.plane_name
leotrack-> ORDER BY anomaly_count DESC,
leotrack->     avg_battery_pct ASC -- при равных аномалиях – сначала худший заряд
leotrack-> LIMIT 5;
  sat_id | norad_id | sat_name | status      | plane_name | anomaly_count | avg_battery_pct
-----+-----+-----+-----+-----+-----+-----
      3 | 58003   | LT-A-03 | operational | Alpha-550  |             5 |            76.5
      6 | 58006   | LT-B-02 | degraded   | Beta-560   |             2 |            77.1
      4 | 58004   | LT-A-04 | operational | Alpha-550  |             1 |            77.1
      2 | 58002   | LT-A-02 | operational | Alpha-550  |             1 |            79.0
      9 | 58009   | LT-G-02 | operational | Gamma-570  |             1 |            81.0

```

Рис. Результат выполнения запроса 6.7

6.8. Суточный трафик по орбитальным плоскостям

Тип запроса: `DATE_TRUNC + многотабличный JOIN`

Что делает этот запрос:

Детализация трафика по дням и орбитальным плоскостям за период март–май 2026. Позволяет выявлять пиковые дни нагрузки и распределение трафика по частям созвездия.

Зачем выбраны именно эти конструкции:

`DATE_TRUNC("day", ds.start_time)::DATE` группирует сессии по дате, отсекая время. Цепочка JOIN из 4 таблиц (`data_sessions` → `satellite_beams` → `satellites` → `orbital_planes`) позволяет агрегировать трафик на уровне орбитальной плоскости, минуя промежуточные уровни детализации.

SQL — запрос 6.8

```
SELECT
op.plane_name,
-- DATE_TRUNC обрезает время, ::DATE убирает временную зону
DATE_TRUNC('day', ds.start_time)::DATE AS traffic_date,
COUNT(*) AS session_count,
ROUND(SUM(ds.bytes_down + ds.bytes_up)
/ 1073741824.0, 3) AS total_gb,
ROUND(AVG(ds.avg_latency_ms), 1) AS avg_latency_ms
FROM data_sessions ds
JOIN satellite_beams sb ON sb.beam_id = ds.beam_id
JOIN satellites sat ON sat.sat_id = sb.sat_id
JOIN orbital_planes op ON op.plane_id = sat.plane_id
WHERE ds.start_time >= '2026-03-01'
AND ds.start_time < '2026-06-01'
GROUP BY op.plane_name,
DATE_TRUNC('day', ds.start_time)
ORDER BY traffic_date, total_gb DESC;
```

```

leotrack=> SELECT
leotrack->     op.plane_name,
leotrack->     -- DATE_TRUNC обрезает время, ::DATE убирает временную зону
leotrack->     DATE_TRUNC('day', ds.start_time)::DATE AS traffic_date,
leotrack->     COUNT(*) AS session_count,
leotrack->     ROUND(SUM(ds.bytes_down + ds.bytes_up)
leotrack->         / 1073741824.0, 3) AS total_gb,
leotrack->     ROUND(AVG(ds.avg_latency_ms), 1) AS avg_latency_ms
leotrack-> FROM data_sessions ds
leotrack-> JOIN satellite_beams sb ON sb.beam_id = ds.beam_id
leotrack-> JOIN satellites sat ON sat.sat_id = sb.sat_id
leotrack-> JOIN orbital_planes op ON op.plane_id = sat.plane_id
leotrack-> WHERE ds.start_time >= '2026-03-01'
leotrack->     AND ds.start_time < '2026-06-01'
leotrack-> GROUP BY op.plane_name,
leotrack->     DATE_TRUNC('day', ds.start_time)
leotrack-> ORDER BY traffic_date, total_gb DESC;
plane_name | traffic_date | session_count | total_gb | avg_latency_ms
-----
Alpha-550 | 2026-03-01 | 1 | 5.500 | 28.0
Alpha-550 | 2026-03-02 | 1 | 3.300 | 25.0
Alpha-550 | 2026-03-03 | 1 | 2.200 | 35.0
Alpha-550 | 2026-03-04 | 1 | 55.000 | 22.0
Alpha-550 | 2026-03-05 | 1 | 110.000 | 20.0
Alpha-550 | 2026-03-06 | 1 | 1.100 | 40.0
Alpha-550 | 2026-03-07 | 1 | 4.400 | 30.0
Alpha-550 | 2026-03-08 | 1 | 3.300 | 27.0
Alpha-550 | 2026-03-10 | 1 | 22.000 | 23.0
Alpha-550 | 2026-03-15 | 1 | 11.000 | 32.0
Alpha-550 | 2026-04-01 | 1 | 6.600 | 28.0
Alpha-550 | 2026-04-02 | 1 | 2.200 | 26.0
Alpha-550 | 2026-04-05 | 1 | 66.000 | 21.0
Alpha-550 | 2026-04-10 | 1 | 33.000 | 22.0
Alpha-550 | 2026-05-01 | 1 | 5.500 | 29.0
Alpha-550 | 2026-05-02 | 1 | 110.000 | 20.0
Alpha-550 | 2026-05-10 | 1 | 4.400 | 31.0
Alpha-550 | 2026-05-17 | 9 | 56.107 | 64.8
Beta-560 | 2026-05-17 | 2 | 6.216 | 43.5
Alpha-550 | 2026-05-18 | 8 | 46.429 | 72.0
Beta-560 | 2026-05-18 | 4 | 26.768 | 36.8
Alpha-550 | 2026-05-19 | 10 | 69.998 | 58.5
Beta-560 | 2026-05-19 | 2 | 10.071 | 85.0
Alpha-550 | 2026-05-20 | 10 | 63.488 | 58.0
Beta-560 | 2026-05-20 | 2 | 7.807 | 64.5
Alpha-550 | 2026-05-21 | 10 | 59.974 | 57.8

```

Рис. Результат выполнения запроса 6.8

6.9. Просроченные и незакрытые счета с расчётом дней просрочки

Тип запроса: `WHERE + вычисляемый столбец + ORDER BY`

Что делает этот запрос:

Отчёт по просроченным и открытым счетам с вычислением количества дней просрочки. Используется биллинговой службой для выставления пеней и автоматической блокировки абонентов.

Зачем выбраны именно эти конструкции:

`(CURRENT_DATE - i.due_date)` — вычисляемый столбец прямо в `SELECT`: разница дат даёт тип `INTERVAL/INTEGER` (количество дней). `WHERE i.due_date < CURRENT_DATE` исключает счета с ещё не наступившим сроком, даже если они в статусе `pending`.

SQL — запрос 6.9

```
SELECT
  s.sub_id,
  s.full_name,
  s.phone,
  s.balance,
  i.invoice_id,
  i.billing_period,
  i.total_amount,
  i.status AS invoice_status,
  -- вычисляемый столбец: сколько дней прошло после дедлайна
  (CURRENT_DATE - i.due_date) AS days_overdue
FROM subscribers s
JOIN invoices i ON i.sub_id = s.sub_id
WHERE i.status IN ('overdue', 'pending') -- незакрытые счета
AND i.due_date < CURRENT_DATE -- только реально просроченные
ORDER BY days_overdue DESC, -- сначала самые старые
s.balance ASC; -- затем наибольший долг
```

```

leotrack=> SELECT
leotrack->     s.sub_id,
leotrack->     s.full_name,
leotrack->     s.phone,
leotrack->     s.balance,
leotrack->     i.invoice_id,
leotrack->     i.billing_period,
leotrack->     i.total_amount,
leotrack->     i.status AS invoice_status,
leotrack->     -- вычисляемый столбец: сколько дней прошло после дедлайна
leotrack->     (CURRENT_DATE - i.due_date) AS days_overdue
leotrack-> FROM subscribers s
leotrack-> JOIN invoices i ON i.sub_id = s.sub_id
leotrack-> WHERE i.status IN ('overdue', 'pending') -- незакрытые счета
leotrack-> AND i.due_date < CURRENT_DATE -- только реально просроченные
leotrack-> ORDER BY days_overdue DESC, -- сначала самые старые
leotrack->     s.balance ASC; -- затем наибольший долг

```

sub_id	full_name	phone	balance	invoice_id	billing_period	total_amount	invoice_status	days_overdue
12	Mikhail Zaitsev	+79160020020	-380.00	1843	2026-03	342.49	pending	56
5	Ivan Sidorov	+79164004004	-50.00	1836	2026-03	341.44	overdue	56
8	Dmitry Kuznetsov	+79167007007	800.00	1839	2026-03	200.69	pending	56
4	Maria Ivanova	+79163003003	2100.00	1835	2026-03	441.90	pending	56
15	Sergey Romanov	+79001000003	319.51	1991	2026-03	185.99	pending	32
16	Andrey Abramov	+79001000004	392.68	1992	2026-03	469.99	overdue	32
20	Petr Volkov	+79001000008	685.36	1996	2026-03	235.99	pending	32
21	Nikolai Belov	+79001000009	758.53	1997	2026-03	299.99	overdue	32
25	Anna Zhao	+79001000013	1051.21	2001	2026-03	625.99	pending	32
26	Olga Huang	+79001000014	1124.38	2002	2026-03	959.99	overdue	32
30	Ekaterina Li	+79001000018	1417.06	2006	2026-03	445.99	pending	32
31	Svetlana Wang	+79001000019	1490.23	2007	2026-03	379.99	overdue	32
35	Robert Kozlov	+79001000023	1782.91	2011	2026-03	275.99	pending	32
36	Michael Lebedev	+79001000024	1856.08	2012	2026-03	399.99	overdue	32
40	Joseph Sidorov	+79001000028	2148.76	2016	2026-03	935.99	pending	32
41	Thomas Petrov	+79001000029	2221.93	2017	2026-03	199.99	overdue	32
45	Sophia Lewis	+79001000033	2514.61	2021	2026-03	355.99	pending	32
46	Isabella Rodriguez	+79001000034	2587.78	2022	2026-03	589.99	overdue	32
50	Amelia Garcia	+79001000038	2880.46	2026	2026-03	375.99	pending	32
51	Harper Thompson	+79001000039	2953.63	2027	2026-03	659.99	overdue	32
55	Yuki Jackson	+79001000043	3246.31	2031	2026-03	175.99	pending	32
56	Hiroshi Thomas	+79001000044	3319.48	2032	2026-03	239.99	overdue	32
60	Sofia Brown	+79001000048	3612.16	2036	2026-03	565.99	pending	32
61	Luca Johnson	+79001000049	3685.33	2037	2026-03	899.99	overdue	32
5	Ivan Sidorov	+79164004004	-50.00	928	2026-04	408.94	pending	25

Рис. Результат выполнения запроса 6.9

6.10. Качество сигнала по активным лучам спутников (SNR)

Тип запроса: `LEFT JOIN + HAVING + SNR-мониторинг`

Что делает этот запрос:

Статистика качества сигнала (Signal-to-Noise Ratio, дБ) для каждого активного луча: среднее и минимальное значения SNR, количество обслуживаемых терминалов. Применяется RF-инженером для выявления лучей с деградацией.

Зачем выбраны именно эти конструкции:

`LEFT JOIN beam_coverage` — берём все лучи, а к ним присоединяем покрытия (`LEFT`, чтобы не потерять лучи без терминалов). Условие `bc.released_at IS NULL` в `JOIN` фильтрует только активные назначения (а не в `WHERE` — иначе `LEFT JOIN` превратился бы в `INNER JOIN`). `HAVING COUNT > 0` оставляет только лучи с терминалами.

SQL — запрос 6.10

```
SELECT
  sat.sat_name,
  sat.norad_id,
  sb.beam_index,
  op.plane_name,
  COUNT(bc.coverage_id) AS terminals_served,
  ROUND(AVG(bc.snr_db), 2) AS avg_snr_db,
  MIN(bc.snr_db) AS min_snr_db
FROM satellite_beams sb
JOIN satellites sat ON sat.sat_id = sb.sat_id
JOIN orbital_planes op ON op.plane_id = sat.plane_id
-- LEFT JOIN + условие в ON (не WHERE!) сохраняет лучи без терминалов
LEFT JOIN beam_coverage bc
ON bc.beam_id = sb.beam_id
AND bc.released_at IS NULL -- только активные назначения
GROUP BY sat.sat_id, sat.sat_name, sat.norad_id,
sb.beam_index, op.plane_name
HAVING COUNT(bc.coverage_id) > 0 -- только лучи с терминалами
ORDER BY avg_snr_db ASC -- сначала худшее качество сигнала
LIMIT 10;
```

```

leotrack=> SELECT
leotrack->     sat.sat_name,
leotrack->     sat.norad_id,
leotrack->     sb.beam_index,
leotrack->     op.plane_name,
leotrack->     COUNT(bc.coverage_id) AS terminals_served,
leotrack->     ROUND(AVG(bc.snr_db), 2) AS avg_snr_db,
leotrack->     MIN(bc.snr_db) AS min_snr_db
leotrack-> FROM satellite_beams sb
leotrack-> JOIN satellites sat ON sat.sat_id = sb.sat_id
leotrack-> JOIN orbital_planes op ON op.plane_id = sat.plane_id
leotrack-> -- LEFT JOIN + условие в ON (не WHERE!) сохраняет лучи без терминалов
leotrack-> LEFT JOIN beam_coverage bc
leotrack->     ON bc.beam_id = sb.beam_id
leotrack->     AND bc.released_at IS NULL -- только активные назначения
leotrack-> GROUP BY sat.sat_id, sat.sat_name, sat.norad_id,
leotrack->     sb.beam_index, op.plane_name
leotrack-> HAVING COUNT(bc.coverage_id) > 0 -- только лучи с терминалами
leotrack-> ORDER BY avg_snr_db ASC -- сначала худшее качество сигнала
leotrack-> LIMIT 10;
  sat_name | norad_id | beam_index | plane_name | terminals_served | avg_snr_db | min_snr_db
-----+-----+-----+-----+-----+-----+-----
LT-A-02 | 58002 | 2 | Alpha-550 | 1 | 17.20 | 17.20
LT-A-02 | 58002 | 1 | Alpha-550 | 1 | 19.80 | 19.80
LT-A-01 | 58001 | 3 | Alpha-550 | 9 | 20.59 | 10.10
LT-A-01 | 58001 | 2 | Alpha-550 | 10 | 21.01 | 13.31
LT-A-01 | 58001 | 1 | Alpha-550 | 4 | 22.30 | 18.50
(5 rows)

```

Рис. Результат выполнения запроса 6.10

7. Пользовательские функции PL/pgSQL (03_functions.sql)

Реализовано 9 функций, автоматизирующих основные операции системы. Функции делятся на скалярные (возвращают одно значение), табличные (возвращают набор строк) и служебные (возвращают число обработанных записей).

Сигнатура функции	Тип	Описание
<code>get_subscriber_balance(p_sub_id INT) → NUMERIC</code>	Скалярная	Возвращает текущий баланс абонента. Возвращает NULL если абонент не найден.
<code>get_subscriber_status_label(p_sub_id INT) → TEXT</code>	Скалярная	Читаемая метка статуса: "Active" / "Blocked by operator" / "Suspended on request" / "Contract cancelled".
<code>classify_balance(p_sub_id INT) → TEXT</code>	Скалярная	Классификация баланса: "Critical debt" (< -1000) / "Debt" / "Low balance" / "Normal" / "High balance".
<code>get_total_data_gb(p_sub_id INT) → NUMERIC</code>	Скалярная	Суммирует весь трафик абонента по всем его терминалам, возвращает гигабайты (байты / 1073741824).
<code>count_anomalies(p_sat_id INT, p_days INT=7) → INT</code>	Скалярная	Количество телеметрических аномалий спутника за последние N дней. По умолчанию 7 дней.
<code>can_subscribe_plan(p_sub_id INT, p_plan_id INT) → BOOLEAN</code>	Скалярная	TRUE если у абонента нет просроченных счетов И план активен (is_active=TRUE). Бизнес-правило смены тарифа.
<code>get_subscriber_finance_card(p_sub_id INT) → TABLE</code>	Табличная	Полная финансовая карточка: план, баланс, статус, число терминалов, число счетов, суммарные платежи.
<code>get_top_revenue_subscribers(p_limit INT=10) → TABLE</code>	Табличная	Топ-N абонентов по суммарному объёму всех платежей. Используется для VIP-программ.
<code>get_overdue_subscribers(p_min_debt NUMERIC=0) → TABLE</code>	Табличная	Абоненты с просроченными счетами, суммарный долг которых превышает p_min_debt. Параметрический фильтр.
<code>recalculate_overage_cost(p_billing_period VARCHAR) → INT</code>	Служебная	Пересчитывает сверхлимитные начисления по всем счетам периода на основе реальных данных сессий. Возвращает число обновлённых строк.

03_functions.sql — примеры вызовов функций (06_demo.sql)

```
-- Полная финансовая карточка абонента #1 (Anisimov)
SELECT * FROM get_subscriber_finance_card(1);

-- Топ-5 абонентов по суммарным платежам
SELECT * FROM get_top_revenue_subscribers(5);

-- Должники с долгом больше 0
SELECT * FROM get_overdue_subscribers(0);

-- Классификация балансов всех абонентов
SELECT sub_id, full_name, balance,
classify_balance(sub_id) AS category
FROM subscribers ORDER BY balance;

-- Проверка допустимости смены тарифа
SELECT
can_subscribe_plan(1, 4) AS owner_can_enterprise, -- TRUE
can_subscribe_plan(5, 3) AS debtor_can_professional; -- FALSE (overdue)
```

```

leotrack=> -- Финансовая карточка абонента #1 (Anisimov)
leotrack=> SELECT * FROM get_subscriber_finance_card(1);
      full_name | email | phone | plan_name | monthly_fee | balance | status | terminals | total_invoices | total_paid
-----
Anisimov Timur Yurievich | tanisimov2006@gmail.com | +79991234560 | Government | 799.99 | 9999.00 | active | 1 | 2 | 4088.75
(1 row)

leotrack=>
leotrack=> -- Топ-5 абонентов по суммарным платежам
leotrack=> SELECT * FROM get_top_revenue_subscribers(5);
  sub_id | full_name | plan_name | total_revenue
-----
1 | Anisimov Timur Yurievich | Government | 4088.75
4 | Maria Ivanova | Enterprise | 3276.48
6 | Nikolai Frolov | Aviation | 3117.29
33 | James Morozov | Government | 2490.87
11 | Yuliya Belova | Enterprise | 2472.49
(5 rows)

```

Рис. Результат вызова `get_subscriber_finance_card(1)` и `get_top_revenue_subscribers(5)`

8. Триггеры (04_triggers.sql)

Реализовано 6 триггеров, защищающих целостность данных и автоматизирующих сервисную логику. Каждый триггер решает конкретную задачу, которую невозможно или нецелесообразно реализовать только на уровне CHECK-ограничений.

Триггер	Событие	Описание и обоснование
<code>trg_protect_balance</code>	BEFORE UPDATE OF balance ON subscribers	Запрещает снижение баланса абонента ниже -5000. Несмотря на наличие CHECK, триггер даёт осмысленное сообщение об ошибке с указанием попытки: "Balance cannot drop below -5000. Attempted value: -9999". CHECK-ограничение срабатывает молча, триггер — информативно.
<code>trg_subscriber_audit</code>	AFTER INSERT OR UPDATE OR DELETE ON subscribers	Ведёт полный журнал изменений в таблице <code>subscriber_audit</code> : тип операции (INSERT/UPDATE/DELETE), старый и новый статус, старый и новый баланс, имя пользователя БД (<code>CURRENT_USER</code>). Обязателен для систем с финансовой ответственностью.
<code>trg_set_updated_at</code>	BEFORE UPDATE ON subscribers	Автоматически обновляет поле <code>updated_at = NOW()</code> при каждом изменении записи абонента. Приложение не должно помнить про этот столбец — триггер делает это прозрачно.

Триггер	Событие	Описание и обоснование
<code>trg_invoice_status_guard</code>	BEFORE UPDATE OF status ON invoices	Запрещает откат статуса оплаченного счёта (<code>paid</code> → <code>pending/overdue</code>). Также запрещает переход <code>overdue</code> → <code>pending</code> (только → <code>paid</code>). Реализует конечный автомат состояний счёта на уровне БД.
<code>trg_archive_invoice</code>	AFTER DELETE ON invoices	При удалении счёта автоматически переносит запись в <code>invoice_archive</code> с временной меткой <code>archived_at</code> . Обеспечивает полный аудит жизненного цикла счёта — ни одна запись не теряется.

<code>trg_protect_active_plan</code>	BEFORE DELETE ON service_plans	Запрещает удаление тарифного плана с флагом <code>is_active = TRUE</code> . Защищает от случайного удаления плана, к которому привязаны абоненты, даже если ON DELETE RESTRICT на <code>subscribers.plan_id</code> не сработал.
<code>trg_log_sat_anomaly</code>	AFTER INSERT ON telemetry	При добавлении телеметрического пакета с <code>anomaly_flag = TRUE</code> автоматически дублирует запись в <code>satellite_health_log</code> . Оперативный журнал аномалий для системы мониторинга — отдельная таблица быстрее, чем каждый раз фильтровать всю <code>telemetry</code> .

06_demo.sql — демонстрация всех триггеров

```
-- -- ТРИГГЕР 1: защита баланса -----
-- Попытка установить баланс -9999 должна завершиться исключением
DO $$
BEGIN
BEGIN
UPDATE subscribers SET balance = -9999.00 WHERE sub_id = 7;
EXCEPTION WHEN others THEN
RAISE NOTICE 'TRIGGER 1 OK: %', SQLERRM;
END;
END;
$$;

-- Ожидаемый вывод: TRIGGER 1 OK: Balance cannot drop below -5000...

-- -- ТРИГГЕР 2: аудит изменений -----
UPDATE subscribers SET balance = balance + 100 WHERE sub_id = 3;
SELECT * FROM subscriber_audit ORDER BY audit_id DESC LIMIT 5;

-- -- ТРИГГЕР 3: автообновление updated_at -----
UPDATE subscribers SET status = 'active' WHERE sub_id = 9;
SELECT sub_id, full_name, updated_at FROM subscribers WHERE sub_id = 9;

-- -- ТРИГГЕР 4: защита статуса счёта -----
-- Попытка вернуть оплаченный счёт в pending — должна упасть
DO $$
BEGIN
BEGIN
UPDATE invoices SET status = 'pending' WHERE invoice_id = 1;
EXCEPTION WHEN others THEN
RAISE NOTICE 'TRIGGER 4 OK: %', SQLERRM;
```



```

END;
END;
$$;

-- -- ТРИГГЕР 5: архивирование при удалении счёта -----
INSERT INTO invoices (sub_id, billing_period, issue_date, due_date,
amount_base, total_amount, status)
VALUES (7, '2026-06', CURRENT_DATE, CURRENT_DATE + 30, 29.99, 29.99, 'pending');
DELETE FROM invoices WHERE sub_id = 7 AND billing_period = '2026-06';
SELECT * FROM invoice_archive ORDER BY archived_at DESC LIMIT 3;

-- -- ТРИГГЕР 7: лог аномалии телеметрии -----
INSERT INTO telemetry (sat_id, recorded_at, battery_pct, anomaly_flag)
VALUES (3, NOW(), 45.0, TRUE);
SELECT * FROM satellite_health_log ORDER BY logged_at DESC LIMIT 3;

```

```

leotrack=> -- Аудит изменений (после UPDATE из триггера 2)
leotrack=> SELECT * FROM subscriber_audit ORDER BY audit_id DESC LIMIT 5;

```

audit_id	sub_id	operation	changed_at	old_status	new_status	old_balance	new_balance	changed_by
22688	9	UPDATE	2026-05-25 10:56:30.682481	active	active	0.00	0.00	tanis
22687	3	UPDATE	2026-05-25 10:56:30.655817	active	active	620.50	720.50	tanis
22686	62	INSERT	2026-05-25 10:56:01.337456		active		3758.50	tanis
22685	61	INSERT	2026-05-25 10:56:01.337456		cancelled		3685.33	tanis
22684	60	INSERT	2026-05-25 10:56:01.337456		suspended		3612.16	tanis

(5 rows)

```

leotrack=>
leotrack=> -- Архив удалённых счетов (после DELETE из триггера 5)
leotrack=> SELECT * FROM invoice_archive ORDER BY archived_at DESC LIMIT 3;

```

archive_id	invoice_id	sub_id	billing_period	total_amount	status	archived_at
3096	2039	7	2026-06	29.99	pending	2026-05-25 10:56:30.707577
2996	1890	81	2026-04	179.99	paid	2026-05-25 10:56:01.337456
2997	1891	81	2026-03	194.99	paid	2026-05-25 10:56:01.337456

(3 rows)

Рис. Результат демо-сценария: *subscriber_audit* и *invoice_archive*

9. Контейнеризация (docker-compose.yml)

Проект разворачивается одной командой `docker compose up -d`. PostgreSQL 16 (alpine-образ) запускается с автоматической инициализацией: при первом старте контейнера все SQL-скрипты из директории `docker-entrypoint-initdb.d/` выполняются в алфавитном порядке (01_schema → 02_data → 03_functions → 04_triggers → 05_queries → 06_demo). Healthcheck на основе `pg_isready` гарантирует готовность БД перед подключением.

`docker-compose.yml` — конфигурация контейнера

```

version: '3.9'

services:
  leotrack_db:
    image: postgres:16-alpine # лёгкий образ (~85 МБ)
    container_name: leotrack_postgres
    restart: unless-stopped # перезапуск при сбое
    environment:
      POSTGRES_DB: leotrack
      POSTGRES_USER: tanis
      POSTGRES_PASSWORD: 123
      TZ: Europe/Moscow
    ports:

```

```

- "5432:5432" # проброс на localhost
volumes:
- leotrack_pgdata:/var/lib/postgresql/data # постоянные данные
# Скрипты инициализации — выполняются однократно при первом старте
- ./sql/01_schema.sql:/docker-entrypoint-initdb.d/01_schema.sql:ro
- ./sql/02_data.sql:/docker-entrypoint-initdb.d/02_data.sql:ro
- ./sql/03_functions.sql:/docker-entrypoint-initdb.d/03_functions.sql:ro
- ./sql/04_triggers.sql:/docker-entrypoint-initdb.d/04_triggers.sql:ro
- ./sql/05_queries.sql:/docker-entrypoint-initdb.d/05_queries.sql:ro
- ./sql/06_demo.sql:/docker-entrypoint-initdb.d/06_demo.sql:ro
healthcheck:
test: ["CMD-SHELL", "pg_isready -U tanis -d leotrack"]
interval: 10s
timeout: 5s
retries: 5

volumes:
leotrack_pgdata:
driver: local # данные хранятся между перезапусками контейнера

```

Команды для работы с контейнером:

Команды управления контейнером

```

# Запустить контейнер (первый раз — с инициализацией БД)
docker compose up -d

# Просмотр статуса и логов инициализации
docker compose ps
docker compose logs leotrack_db

# Подключиться к psql внутри контейнера
docker exec -it leotrack_postgres psql -U tanis -d leotrack

# Полный сброс данных (ВНИМАНИЕ: безвозвратно удаляет volume!)
docker compose down -v

```

```

PS C:\Users\tanis\Documents\leotrack_db> docker compose up -d
time="2026-05-26T00:47:18+07:00" level=warning msg="C:\\Users\\tanis\\Documents\\leotrack_db\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 1/1
✔Container leotrack_postgres Running

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\tanis\Documents\leotrack_db>
PS C:\Users\tanis\Documents\leotrack_db> # Проверка статуса
PS C:\Users\tanis\Documents\leotrack_db> docker compose ps
time="2026-05-26T00:47:20+07:00" level=warning msg="C:\\Users\\tanis\\Documents\\leotrack_db\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
leotrack_postgres	postgres:16-alpine	"docker-entrypoint.s..."	leotrack_db	14 hours ago	Up 11 seconds (healthy)

```

0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
PS C:\Users\tanis\Documents\leotrack_db>

```

Рис. Вывод docker compose up -d и docker compose ps

10. Заключение

В рамках расчётно-графической работы спроектирована и реализована реляционная база данных LEO TRACK — система мониторинга и биллинга спутникового интернет-сервиса. Выполнены все требования уровня «отлично» (Вариант А — Docker + Вариант В - PL/pgSQL/триггеры):

Категория	Выполненная работа
Схема БД	12 таблиц с полным набором ограничений: NOT NULL, UNIQUE, CHECK, FOREIGN KEY с явными правилами CASCADE/RESTRICT/SET NULL. Все ограничения обоснованы бизнес-логикой предметной области.
Индексация	17+ индексов, включая составные (terminal_id, start_time DESC) и partial (WHERE anomaly_flag = TRUE). Стратегия выбора обоснована полями с высокой селективностью.
SQL-запросы	10 запросов покрывают: простые SELECT, WHERE, JOIN (INNER/LEFT), GROUP BY, HAVING, ORDER BY/LIMIT, CTE (WITH), DATE_TRUNC, COUNT FILTER, вычисляемые столбцы.
Функции PL/pgSQL	9 функций: скалярные (get_subscriber_balance, classify_balance), табличные (get_subscriber_finance_card, get_top_revenue_subscribers), служебная (recalculate_overage_cost).
Триггеры	6 триггеров: защита баланса, аудит изменений абонентов, автообновление updated_at, конечный автомат статусов счётов, архивирование при удалении, лог аномалий телеметрии.
Docker	Контейнеризация с PostgreSQL 16-alpine, автоматической инициализацией всех скриптов и healthcheck на pg_isready.
Тестовые данные	Более 100 записей суммарно: разнообразные статусы, отрицательные балансы, просроченные счета, телеметрические аномалии — для репрезентативных результатов запросов.
README.md	Полная документация: описание проекта, инструкции запуска, подключения через psql/DBBeaver, таблица сущностей, обоснование ограничений, команда полного сброса.

